Claude Code vs CoStrict 深度对比分析

> 报告日期：2026-04-02
> 代码依据：Claude Code 反编译版 + CoStrict CLI v3.0.18

目录

1. Claude Code 深度分析以及优缺点
2. CoStrict 深度分析以及优缺点
3. Claude Code & CoStrict 深入各个维度的对比
4. CoStrict+开源模型 比 claude code+开源模型 更强？
5. Claude Code 源码泄漏事件后，CoStrict 应该怎么做

一、Claude Code 深度分析以及优缺点

1.1 核心技术栈

层级	技术选型	说明
运行时	Bun	非 Node.js；原生 TypeScript，无需 tsc 编译
语言	TypeScript + TSX	`strict: false`，反编译产物存在 ~1341 个 tsc 错误
构建产物	25MB 单文件 Bundle	`bun build --target bun`，零运行时依赖
终端 UI	React 19 + Ink	`react-reconciler` 自定义渲染器驱动终端
LLM 接入	`@anthropic-ai/sdk ^0.80.0`	直连 Anthropic；另有 Bedrock / Vertex / Foundry SDK
Schema 验证	Zod v4	工具输入、配置、权限规则统一验证
并发控制	`p-map ^7.0.4`	工具并发执行的并发数限制
扩展协议	`@modelcontextprotocol/sdk ^1.29.0`	MCP 标准 SDK

1.2 核心架构

1.2.1 Agent Loop：ReAct 状态机

Agent Loop 是所有能力的执行引擎，位于 `src/query.ts`（1732 行）。核心是一个 `while(true)` 状态机，每轮执行 ReAct 三步：

Reason：LLM 流式输出，遇到 `tool_use` block 立即调度（不等完整输出）

Act：执行工具，每个 `tool_result` 对应一个 `tool_use`，通过 `tool_use_id` 严格关联

Observe：工具结果作为新 `user` 消息追加，下一轮 LLM 可见

循环控制由 `transition.reason` 字段驱动：

transition.reason	触发条件	下一步动作
`next_turn`	工具执行完	正常进入下一轮
`collapse_drain_retry`	首次遇到 PTL	尝试 context collapse
`reactive_compact_retry`	collapse 失败	执行反应式压缩
`max_output_tokens_escalate`	首次 max_tokens	升级到 64k
`max_output_tokens_recovery`	升级后仍不够	最多重试 3 次
`stop_hook_blocking`	stop hook 阻塞错误	注入错误消息重查询
`token_budget_continuation`	token budget 要求继续	注入 nudge 消息

1.2.2 流式执行引擎与工具调度

`StreamingToolExecutor` + `toolOrchestration.ts` 实现了精细的并发调度：

```
// LLM 输出第一个 tool_use block → 立即调度，不等完整输出
// isConcurrencySafe=true 的连续工具 → 合并为一批并行执行
// 例：[Read, Read, Write, Read] → [{Read,Read}, {Write}, {Read}]
// 最大并发数：CLAUDE_CODE_MAX_TOOL_USE_CONCURRENCY = 10（默认）
// 某工具失败 → siblingAbortController 立即取消同批次其他任务
```

`partitionToolCalls` 分区逻辑：按 `isConcurrencySafe` 分区，连续安全工具合并为同一批次，非安全工具单独成批。

1.2.3 工具体系

工具是携带调度元数据的执行节点，核心字段：

- `isConcurrencySafe(input)` → 决定是否可以并发执行
- `isReadOnly(input)` → 影响权限路由
- `isDestructive(input)` → 影响用户确认提示
- `maxResultSizeChars` → 超过此阈值写磁盘（默认 50k），`tool_result` 只返回摘要

工具注册在 `src/tools.ts`，按条件组装。部分工具（`TaskCreate/Get/Update/List/Output/Stop` 等）通过 feature flag 动态加载。

1.2.4 五层上下文管理

```
层一：静态记忆（CLAUDE.md）→ 零 token 消耗，不随压缩丢失
      优先级：企业托管 < 用户全局 < 项目根 < 规则文件集 < CLAUDE.local.md

层二：动态触发阈值
      有效窗口 = contextWindow - min(maxOutputTokens, 20k)
      触发阈值 = 有效窗口 - 13k
```

```
        电路断路器：连续失败 3 次停止重试

层三：双路压缩执行
        Session Memory（精细）：保留最近 10k~40k tokens 原始消息 + 早期历史 AI 摘要
        传统 9 段摘要（兜底）：Primary Request / Key Concepts / Files / Errors /
                                Problem Solving / User Messages / Pending Tasks /
                                Current Work / Next Step

层四：Microcompact → 零 LLM 调用，清除旧工具输出内容，计数/时间触发

层五：compact_boundary 标记 → 每次压缩后插入系统消息记录元数据，
        QueryEngine 看到后释放旧消息供 GC 回收，防止进程内存无限增长
```

1.2.5 任务系统：跨 Agent 的共享协调面

六个任务工具（`TaskCreate / TaskGet / TaskUpdate / TaskList / TaskOutput / TaskStop`）构成完整的协调基础设施：

- **文件锁原子认领**：`proper-lockfile` 保证并发 Agent 不重复执行同一任务
- **依赖图调度**：`blockedBy` 字段声明任务依赖，`claimTask` 检查依赖未完成则拒绝认领
- **DiskTaskOutput**：异步任务输出写临时文件，最大 5GB，`O_NOFOLLOW` 防 symlink 攻击
- **Agent 任务移交**：Agent 退出时 `unassignTeammateTasks` 将其未完成任务重置为 `pending` 等待接管
- **通知机制**：每秒轮询，完成后以 XML 格式注入 Agent 的 user 消息

任务存储在 `~/.config/claude/tasks/[taskListId]/`，每个任务一个 JSON 文件，`.highwatermark` 文件防止 ID 重用。

1.2.6 权限系统

`src/utils/permissions/permissions.ts`（1486 行）定义六种权限模式：

```
'default'           // 标准提示
'acceptEdits'       // 文件修改自动通过，Shell 仍需确认
'bypassPermissions' // 全自动 YOLO 模式
'dontAsk'           // 全部自动拒绝（只读模式）
'plan'              // 写操作返回预览而非执行，用户确认后切换 acceptEdits
'auto'              // AI 自动分类危险等级（ANT 实验性）
```

`plan` 模式：AI 先跑只读预览，用户确认后再真正执行。支持路径通配符规则（`allow/deny`），规则来源按优先级叠加（`settings.json` → CLI 参数 → 会话级 → 单次命令）。

Bash 安全检查：`bashSecurity.ts`（2592 行）静态检测 23 类 prompt injection 模式（Unicode 混淆、Zsh 特有攻击、`$IFS` 注入、控制字符等）。注：`bashClassifier` 最强部分在反编译版中是 ANT-ONLY stub。

1.2.7 多 Agent 协调

三种执行模式：

- **fork**：继承父会话上下文，同步等待结果
- **async**：后台运行，主流程继续
- **background**：完全独立进程

SendMessageTool 四种通信方式：点对点 / 广播（`to: "*"`）/ Unix Domain Socket / Bridge（远程，ANT-ONLY）

1.2.8 MCP

`src/services/mcp/client.ts`（3351 行）支持 5 种传输协议（Stdio / SSE / HTTP / WebSocket / SDK Control），MCP 工具的 `readOnlyHint` 注解自动映射为 `isConcurrencySafe`，外部 server 声明只读后自动获得并发执行权限。OAuth 主动刷新（过期前 5 分钟），XAA 静默令牌交换，Step-up 认证（403 `insufficient_scope`）。

1.2.9 Skill 系统

Skill 有两种执行模式：

- **inline**：内容注入当前对话上下文，影响后续 AI 行为
- **fork**：通过 `runAgent()` 在独立子代理里运行，有独立 token 预算，完成后结果汇报给父代理

`disableModelInvocation` 标志限制某个 Skill 只能用户手动调用，AI 不能主动触发。

1.2.10 bun:bundle 特性门控

`feature('FLAG_NAME')` 在构建时变成常量，`false` 分支被 tree-shaking 完全消除。ANT-ONLY 功能在发布包中物理不存在：

特性标志	功能
`COORDINATOR_MODE`	多 Agent 协调/Swarm
`BRIDGE_MODE`	IDE 远程控制桥接
`REACTIVE_COMPACT`	反应式上下文压缩
`CONTEXT_COLLAPSE`	上下文折叠（PTL 恢复）
`TOKEN_BUDGET`	Token 预算控制
`VOICE_MODE`	语音输入
`EXTRACT_MEMORIES`	自动记忆提取
`DAEMON`	后台守护进程
`PROACTIVE`	主动建议模式

反编译版里所有 `feature()` 返回 `false`，正式版的完整能力远超文档所见。

1.3 架构亮点

① 五层上下文管理：静态注入（CLAUDE.md 零 token）→ 动态阈值 → 双路压缩执行 → Microcompact 零 LLM 清理 → compact_boundary GC 集成，专为长任务设计，信息损失可控可预测。

② **工具元数据驱动的调度架构**：每个工具的 `isConcurrencySafe / isReadOnly / isDestructive / maxResultSizeChars` 直接驱动三条执行路径：并发调度、权限路由、结果持久化。

③ **流式执行引擎**：LLM 输出第一个 `tool_use` block 时立即调度，状态机 `queued → executing → completed → yielded` 保证结果按序归还，感知延迟最低。

④ **任务系统**：外部存储（文件系统）不受上下文压缩影响，文件锁原子认领防并发冲突，`blockedBy` 依赖图声明任务顺序，Agent 退出后任务自动移交。

⑤ **MCP 工程级实现**：工具注解自动映射并发权限，OAuth 主动管理，工具池排序稳定提高 Prompt Cache 命中率。

⑥ **权限是设计出发点**：六种权限模式覆盖全谱，`plan` 模式预览执行，`bashClassifier` 静态分析危险模式。

⑦ **Async Generator 线性可读性**：整个 Agent Loop 基于 `async function*`，复杂状态机读起来像同步代码，可审计可调试。

⑧ **bun:bundle 构建时特性隔离**：ANT-ONLY 功能物理不存在于发布包，不影响包体积。

1.4 架构缺点

① **绑死 Anthropic**：`providers.ts` 是 41 行 if-else 硬编码，接入 GPT/Gemini/DeepSeek 需改核心代码，无插件化路径。商业模式决定，不会改变。

② **零测试覆盖**：`CLAUDE.md` 明确写着 "No test runner is configured"。1486 行权限系统、1732 行 Agent Loop、3351 行 MCP 客户端，一个测试都没有。

③ **TUI 性能天花板**：`REPL.tsx` 超 5000 行，消息超过 1000 条时帧率下降，React Compiler 的 memoize 输出满屏 `_c(N)` 调用让代码难以调试。

二、CoStrict 深度分析以及优缺点

2.1 核心技术栈

层级	技术选型	说明
运行时	Bun	原生 TypeScript，bun:sqlite 内置驱动
语言	TypeScript（strict 模式）	完整类型覆盖，Zod v4 验证全链路
LLM 接入	Vercel AI SDK（`ai` 包）	统一 `streamText` / `LanguageModelV2` 适配层
Provider 数量	20+	Anthropic/OpenAI/Azure/Bedrock/Vertex/OpenRouter/Groq/Mistral/Cohere/DeepInfra/Cerebras/Together/Perplexity/Vercel/xAI/GitLab/GitHub Copilot 等
终端 UI	React 19 + Ink	TUI 层
桌面端	Electron（`packages/desktop-electron`）	原生桌面应用
存储	SQLite（bun:sqlite）+ Drizzle ORM	结构化持久化
事件总线	Bus 模块（`src/bus/`）	全局事件解耦，TypeScript 泛型强类型
树语法分析	`web-tree-sitter`	FileOutlineTool 代码结构提取
Agent 通信	`@agentclientprotocol/sdk ^0.14.1`	ACP 跨 Agent 通信协议

2.2 核心架构

2.2.1 Agent Loop：三层分离架构

```
SessionPrompt.run()          ← 外层循环：会话状态机（prompt.ts，~1998行）
  ↓ 委托给
SessionProcessor.create()    ← 中层：单次 LLM 调用的生命周期（processor.ts，489行）
  ↓ 调用
LLM.stream()                 ← 底层：跨 Provider 流式接入（llm.ts，351行）
```

外层循环从 SQLite 加载消息，根据 processor 返回的信号决定下一步：

返回值	含义	外层响应
`"continue"`	工具调用完成	继续下一轮
`"stop"`	权限拒绝 / 完成 / 错误	退出循环
`"compact"`	触发压缩阈值	创建压缩标记后继续

Doom Loop 保护：最近 3 个 Part 完全相同的工具调用 → 触发 `doom_loop` 权限询问，阻断无限循环而不是直接报错。

错误重试：最多 10 次，2 秒起步指数退避（最大 30s），优先读响应头的 `retry-after`，`ContextOverflowError` 不重试直接触发压缩。

2.2.2 Provider 层：真正的模型无关

`ProviderTransform`（`src/provider/transform.ts`，1042 行）统一处理所有 Provider 格式差异：

```
// Anthropic 拒绝空 content 消息 → 自动过滤
// Mistral：toolCallId 强制 9 位字母数字 → 自动截断/补零
const normalizedId = part.toolCallId.replace(/[^a-zA-Z0-9]/g, "").substring(0,
9).padEnd(9, "0")
// Claude：toolCallId 只允许 [a-zA-Z0-9_-] → 自动替换非法字符
toolCallId.replace(/[^a-zA-Z0-9_-]/g, "_")
// LiteLLM：注入虚拟 _noop 工具绕过空工具列表兼容问题
```

覆盖 19 个 Provider（对比 Claude Code 的 41 行 if-else）。切换模型时，上层 Agent Loop 感知不到任何差异。

模型能力元数据（`models.ts`）：

```
{
  id: "deepseek-r1",
  reasoning: true,        // 支持推理模式
  tool_call: true,        // 支持工具调用
  attachment: false,      // 不支持图片附件
  interleaved: { field: "reasoning_content" },
  cost: { input: 0.14, output: 2.19 },
  limit: { context: 128000, output: 8000 },
}
```

Agent 发送请求前查询 `model.capabilities`，不向不支持工具调用的模型发送工具定义。

2.2.3 工具体系

CoStrict 专属工具（`costrict/tool/` 目录）：

工具	文件	行数	状态
`SequentialThinkingTool`	`sequential-thinking.ts`	207 行	正式启用
`FileOutlineTool`	`file-outline.ts`	316 行	正式启用
`CheckpointTool`	`checkpoint.ts`	—	experimental
`WorkflowTool`	—	—	正式启用
`CallGraphTool`	`call-graph.ts`	337 行	**已弃用**
`FileImportanceTool`	`file-importance.ts`	331 行	**已弃用**

`FileOutlineTool` 用 Tree-sitter AST 提取类/函数/方法签名，支持 Python/JavaScript/TypeScript/Go/Java/C/C++，对 2000 行 TypeScript 文件：500 token / <100ms / 99% 准确率（vs 全量读取的 4000 token / 2-5s / 85-90%）。

Agent 异构模型支持（`agent/agent.ts`）：

```
model: z.object({
  modelID: ModelID.zod,        // 任意模型 ID
  providerID: ProviderID.zod, // 任意 provider
}).optional(),
model_prompts: z.record(z.string(), z.string()).optional(), // 模型专属 prompt
```

对比 Claude Code 的 `z.enum(['sonnet', 'opus', 'haiku'])`，CoStrict 允许每个子 Agent 指定不同 provider + model，构建多模型混合工作流。

2.2.4 三路上下文压缩

```
策略一：Tool Output Prune（最轻量，零 LLM）
  去除旧工具调用输出，保留最近 40k token 的工具调用输出
  常量：PRUNE_MINIMUM = 20,000 / PRUNE_PROTECT = 40,000

策略二：Session Compaction（一次 LLM 调用）
  触发：isOverflow() 返回 true
  压缩 Agent 生成五段式摘要：
    ## Goal / ## Instructions / ## Discoveries / ## Accomplished / ## Relevant
files
  支持 compaction replay（overflow 时尝试重放上一条用户消息）

策略三：Overflow Compaction（极端场景）
  去除媒体附件，重置到更早消息点，通知用户媒体过大
```

压缩阈值：`config.compaction.reserved`（默认 20k），可配置。

2.2.5 权限系统 PermissionNext

三级动作 + Glob 规则，比 Claude Code 简洁：

```
export const Action = z.enum(["allow", "deny", "ask"])
```

默认规则：`.env` 文件询问，`doom_loop` 必须询问，子 Agent 禁止发起 `question` 和 `plan_enter`。用户选择 "always allow" 时规则写入 SQLite 跨会话持久化。

2.2.6 Plugin 系统

9 个生命周期 Hook 点，内置 6 个 Plugin（CodexAuth / CopilotAuth / CoStrictAuth / TDD / GitlabAuth / Learning），支持 npm 包动态安装和 `file://` 本地插件。

`tool.execute.before` 是最强力的 Hook：可修改工具参数或直接阻断执行，不侵入 Agent Loop 核心代码。

2.2.7 SQLite 持久化存储

核心表：`SessionTable / MessageTable / PartTable / TodoTable / PermissionTable`

Part 分片设计（13 种 Part 类型）：每个 LLM 输出被拆成细粒度 Part 写入 DB。带来三个内存状态下无法实现的能力：

- **Session Fork**：任意消息节点分叉出新会话（`session.fork()`）
- **实时渲染**：通过 `Bus.publish(Event.PartDelta)` 增量更新 UI
- **压缩精度**：`time.compacted` 字段精确标记哪些 Part 已被摘要，粒度更细

2.2.8 Snapshot + Checkpoint 双重安全网

- **Snapshot**：每个 LLM step 开始前自动创建 git diff 快照（不是全量复制），磁盘剩余低于 5GB 时自动禁用，7 天自动清理
- **CheckpointTool**（experimental）：`commit / list / show_diff / restore / revert` 五种 git 操作，模型可主动调用的版本管理工具

2.2.9 LSP 深度集成

完整 client/server 双端实现（`lsp/client.ts` 251 行 + `lsp/server.ts` 486 行）。自动下载管理 4 种 LSP Server（clangd/JDTLS/rust-analyzer/gopls）。

代码理解场景	Claude Code	CoStrict
"这个函数返回类型是什么？"	模型猜测（可能幻觉）	LSP.hover 精确返回
"修改这个参数影响哪里？"	GrepTool 文本搜索	LSP.references 精确
"这个接口有哪些实现？"	模型逐文件猜测	LSP.implementations
重命名变量	文本替换（可能误伤）	LSP.rename 语义级
编辑后有无语法错误	无（运行时发现）	LSP.diagnostics 实时

2.3 核心差异化：17 个工作流 Agent

这是 CoStrict 与 Claude Code 最本质的区别——把软件工程方法论固化为可执行流水线。

工作流矩阵全景：

```
Strict 顶层编排（2 个）
  strict-spec    ← 规格驱动：需求→设计→任务拆分→执行 四阶段严格编排
  strict-plan    ← 计划驱动：探索→澄清→提案→实施 完整生命周期

Spec 规格子工作流（5 个）
  spec-requirement  ← 需求分析（5W2H + 三层目标挖掘）
  spec-design       ← 架构设计（C4 Model 方法论）
  spec-task         ← 任务拆分（单文件级别可验证任务）
  spec-plan         ← 规格计划
  spec-plan-manager ← 计划执行协调

Plan 规划执行子工作流（5 个）
  plan-quick-explore ← 快速探索（漏斗式收敛 + Git 历史挖掘）
  plan-apply         ← 任务执行协调（本身不写代码，只分发）
  plan-sub-coding    ← 代码实现（SubCodingAgent）
  plan-task-check    ← 任务验收
  plan-fix-agent     ← 修复（ReviewAndFix，只修改 task.md，代码委托子 Agent）

TDD 测试驱动工作流（4 个）
  tdd              ← TestDrivenDevelopment 编排 Agent
  run-and-fix      ← 编译/构建/类型检查 + 自动修复（最多 3 轮）
  test-design      ← 测试用例设计（正常/边界/异常三类场景）
  test-and-fix     ← 测试执行 + 自动修复（优先修业务代码，最多 3 轮）

Wiki 文档生成工作流（4 个）
  01-project-analyze / 02-catalogue-design / 03-document-generate / 04-index-generation
```

StrictSpec 权限白名单强制执行六层流水线（来自 `strict-spec.txt`）：

```yaml
permission:
  task:
    general: deny
    explore: deny
    SubCodingAgent: deny     # 禁止直接跳到写代码
    Requirement: allow       # 允许：需求分析
    DesignAgent: allow       # 允许：架构设计
    TaskPlan: allow          # 允许：任务拆分
    PlanManager: allow       # 允许：执行协调
```

StrictPlan 三条铁律（来自 `strict-plan.txt`）：探索驱动澄清 + 项目信息优先 + 代码可答则不问（详见第四章 4.2）

2.4 架构亮点

① **17 个工作流 Agent**：软件工程方法论固化为可执行流水线，开源模型无法绕过流程，不依赖模型自律能力。

② **ProviderTransform**：1042 行统一消化 20+ Provider 格式怪癖，切换模型零代码改动。

③ **SQLite 持久化**：Session Fork、Part 分片精确压缩、进程崩溃可恢复——这三个能力在内存状态下无法实现。

④ **Snapshot + Checkpoint 双重安全网**：无声的自动安全网 + 模型可主动调用的版本管理工具，两层覆盖。

⑤ **LSP client/server 完整实现**：代码诊断不依赖模型推断，自动下载 LSP server，编辑后实时反馈。

⑥ **Bus 事件架构**：`tool.execute.before` 可修改参数或阻断执行，零侵入扩展，6 个内置 Plugin 都用这套机制实现。

⑦ **7 层配置优先级 + `.well-known/opencode` 组织级默认**：企业级配置管控能力完整。

2.5 架构缺点

① **流式工具调度失去精细控制**：委托给 Vercel AI SDK `streamText(maxSteps: 100)`，调度时机由 SDK 内部控制，无法做 Claude Code 那样的 `partitionToolCalls` 精细分区，感知延迟更高。

② **Provider 长尾适配质量不均**：ProviderTransform 主要覆盖 5 个主流 Provider，20+ 里的长尾（Cerebras/DeepInfra/Together AI）edge case 覆盖不足。

③ **运行时特性门控暴露代码**：特性代码在包中完整存在，只通过 `config.experimental.*` 禁用，包体积无法优化，高级用户可直接启用未正式发布的功能。

④ **三路压缩对用户不透明**：无 Claude Code 的 `compact_boundary` 系统消息，用户不知道"这次压缩丢了什么"。使用小窗口开源模型时压缩频率高，不透明性问题被放大。

⑤ **TUI 与 Desktop 能力差距**：文件树、模型选择 GUI、Provider 配置向导只在 Desktop 可用，纯终端用户配置依赖手动编辑 JSON。

三、Claude Code & CoStrict 深入各个维度的对比

3.1 核心对话引擎

两者架构哲学差异明显：Claude Code 用单体状态机驱动，CoStrict 用三层分离架构。

CoStrict 三层分离：

```
SessionPrompt.run()          ← 外层循环：会话状态机（prompt.ts ~1998行）
  ↓ 委托
SessionProcessor.create()  ← 中层：单次 LLM 调用生命周期（processor.ts 489行）
  ↓ 调用
LLM.stream()                ← 底层：跨 Provider 流式接入（llm.ts 351行）
```

外层循环从 SQLite 加载消息，根据 processor 返回的 `"continue" | "stop" | "compact"` 信号决定下一步，状态持久化在数据库而非内存。

3.2 上下文压缩策略

Claude Code 的五层叠加压缩体系：

```
层一：静态记忆（CLAUDE.md）
  会话开始前注入系统提示，零 token 消耗，每次请求都在，不随压缩丢失
  优先级：企业托管 < 用户全局 < 项目根 < 规则文件集 < CLAUDE.local.md

层二：触发机制（autoCompact 动态阈值）
  有效窗口 = contextWindow - min(maxOutputTokens, 20_000)
  触发阈值 = 有效窗口 - 13,000
```

```
  例：Claude 3.5（200k 上下文）→ 阈值 ≈ 167k
  电路断路器：连续失败 3 次停止重试

层三：压缩执行（两路策略，优先走 Session Memory）
  Session Memory 压缩（精细）：
    保留最近 10k~40k tokens 的原始消息
    对早期历史生成 AI 摘要
    → 当前工作上下文保真度高
  传统压缩（兜底，9 段结构化摘要）：
    Primary Request / Key Concepts / Files / Errors /
    Problem Solving / User Messages / Pending Tasks /
    Current Work / Next Step

层四：Microcompact（零 LLM 调用，轻量清理）
  清除旧工具输出内容（保留消息结构）
  计数触发 或 时间触发（距上次操作超过 N 分钟）
  适合"读了很多文件但内容不再重要"的场景

层五：压缩边界标记（compact_boundary）
  每次压缩后插入系统消息，记录压缩元数据：
    触发方式 / token 数量 / 保留段范围 / 已发现工具列表
  QueryEngine 看到边界后释放压缩前消息供 GC 回收
  防止进程内存随对话长度无限增长

PTL 恢复链（遭遇 Prompt Too Long 时的 State 机器路径）：
  collapse_drain_retry（一次）→ 尝试释放 CONTEXT_COLLAPSE 标记之前消息
  reactive_compact_retry → 执行 Reactive Compact（需要 REACTIVE_COMPACT flag，ANT-ONLY）
  两步均失败 → return { reason: 'prompt_too_long' }

max_tokens 恢复链（LLM 输出 token 不足时）：
  max_output_tokens_escalate（一次）→ 8k 升级到 64k
  max_output_tokens_recovery（最多 3 次）→ 注入"Resume from where you left off"恢复消息
  超出限制 → yield 错误消息后 return { reason: 'completed' }
```

CoStrict 的三路压缩策略：

```
策略一：Tool Output Prune（零 LLM 调用，每轮自动）
  PRUNE_PROTECT = 40,000   // 保护最近 40K tokens 的工具调用输出
  PRUNE_MINIMUM = 20,000   // 最小修剪目标
  保护列表：["skill"] — skill 工具调用永不清除

策略二：Session Compaction（一次 LLM 调用）
  触发：isOverflow() → true
  摘要五段式：Goal / Instructions / Discoveries / Accomplished / Relevant files
  replay 机制：压缩后尝试重放最后一条用户消息

策略三：Overflow Compaction（极端场景）
  去除媒体附件（stripMedia: true）
  重置到更早消息点，通知用户
```

各维度明细：

维度	Claude Code	CoStrict
压缩层次	五层叠加（静态注入 / 触发 / 执行 / 轻量清理 / 边界标记）	三路策略（Prune / Compaction / Overflow）
触发阈值计算	`contextWindow - min(maxOutputTokens, 20k) - 13k`（动态公式）	`model.limit.input - config.compaction.reserved`（可配置）
保护逻辑	无工具级保护（Session Memory 保留最近 10k~40k 原始消息）	skill 工具调用永久保护，不被 Prune 清除
轻量清理	Microcompact：零 LLM，清除旧工具输出内容，计数/时间触发	Tool Output Prune：同类功能，每轮自动执行
压缩策略精细度	Session Memory（精细）优先，传统 9 段摘要（兜底）	统一五段式摘要（Goal/Instructions/Discoveries/Accomplished/Files）
回放机制	无	压缩后自动重放最后用户消息
断路器	连续 3 次失败后禁用 autocompact	无
压缩可见性	compact_boundary 标记（记录触发方式、token 量、保留段范围、工具列表）	无等效可见性机制
GC 集成	compact_boundary 触发 QueryEngine 释放旧消息内存	SQLite 持久化，无内存 GC 需求
PTL 恢复	collapse_drain → reactive_compact（ANT-ONLY） → prompt_too_long	ContextOverflowError → compact 信号 → processor 重启
摘要模型	使用当前会话模型	可独立配置专用 compaction agent（不同模型/provider）

3.3 工具系统

3.3.1 工具清单对比

工具类别	Claude Code	CoStrict	谁更强
文件读写	FileRead / FileWrite / FileEdit	read / write / edit / multiedit	CoStrict（批量编辑）
文件搜索	GlobTool / GrepTool	glob / grep / codesearch	CoStrict（语义搜索）
Shell 执行	BashTool（沙箱+权限）	bash（Tree-sitter 路径语义分析）	Claude Code 有沙箱，CoStrict 有语义分析
Web 能力	WebFetch / WebSearch	webfetch / websearch	对等
代码结构提取	无	file-outline（Tree-sitter AST）	CoStrict
思维工具	无（依赖模型原生推理）	sequential-thinking（结构化分支）	CoStrict
MCP 集成	MCPTool（动态生成）	mcp（3种传输，OAuth）	CoStrict（更完整）
多智能体	AgentTool（fork/async/background）	task（子任务）	Claude Code 更成熟
任务管理	TaskCreate/Get/Update/List/Output/Stop	task / todo	Claude Code 更完整
Git 集成	无专用（通过 Bash）	checkpoint（experimental）+ snapshot	CoStrict
Worktree	EnterWorktree / ExitWorktree	src/worktree/（Session 深度集成）	对等（实现方式不同）
Jupyter	NotebookEdit	无	Claude Code
计划模式	EnterPlanMode / ExitPlanMode	plan	对等
技能系统	SkillTool（斜杠命令）	skill	对等
LSP 集成	无	lsp（完整语言服务层）	CoStrict
用户问答	AskUserQuestion（多问题）	question	Claude Code（更丰富选项）
补丁应用	无	apply_patch	CoStrict
批量操作	无	batch	CoStrict
工作流注入	无	workflow（build/plan/spec 三模式）	CoStrict

3.3.2 工具执行并发机制

Claude Code 的并发调度完全自己实现，精细控制调度时机（LLM 输出第一个 tool_use block 就开始执行，不等完整输出）。

CoStrict：把多步工具执行**委托给 Vercel AI SDK 的** `streamText(maxSteps: 100)`，工具调度时机由 SDK 控制

3.3.3 CoStrict 独有工具详解

sequential-thinking

```
interface ThoughtData {
  thought: string
  thoughtNumber: number
  totalThoughts: number        // 可动态增加
  nextThoughtNeeded: boolean
  isRevision?: boolean         // 🔄 修订之前的步骤
  revisesThought?: number
  branchFromThought?: number   // 🌿 从某步分叉
  branchId?: string
}
// 全局思考历史（会话级 thoughtHistory[]）
// 三种渲染格式：💭 Thought / 🔄 Revision / 🌿 Branch
```

file-outline

基于 `web-tree-sitter` 的代码结构提取，AST 解析而非模型推理。

checkpoint

```
支持 5 种 git 操作：commit / list / show_diff / restore / revert
相比 Snapshot（自动 git diff），Checkpoint 提供显式的版本控制点
启用条件：config.experimental.checkpoint !== false
```

3.4 权限系统

Claude Code：

```
规则解析（PermissionRule.ts）
  ↓ 来源分级：settings / CLI / session / command（5 个来源）
规则匹配（路径通配符 + 工具前缀前置过滤）
  ↓ 异步分类器检查（bashClassifier AST 静态分析，ANT-ONLY）
Hook 触发（executePermissionRequestHooks）
  ↓
模式路由（6 种模式）
  ↓
拒绝追踪（连续 3 次拒绝 → 触发 YOLO 提示）
```

Claude Code 6 种权限模式：

```
'default'           // 标准提示（大多数情况）
'acceptEdits'       // 文件修改自动通过，Shell 仍需确认
'bypassPermissions' // 全自动，不问任何问题（YOLO 模式）
'dontAsk'           // 全部自动拒绝（只读模式）
'plan'              // 计划模式：写操作返回预览而非执行
'auto'              // AI 自动分类危险等级（ANT 实验性）
```

CoStrict：

```
// 三种 action + Glob 规则
export const Action = z.enum(["allow", "deny", "ask"])

// 默认规则（agent.ts）
PermissionNext.fromConfig({
  "*":                  "allow",
```

```
  doom_loop:           "ask",
  external_directory:  { "*": "ask", [skillDirs]: "allow" },
  question:            "deny",   // 子 agent 不能发起问题
  plan_enter:          "deny",   // 子 agent 不能进入计划模式
  read: { "*": "allow", "*.env": "ask", "*.env.*": "ask" },
})

// Session 级规则持久化到 SQLite PermissionTable
// 用户选择 "always allow" 后下次同项目直接放行
```

CoStrict Bash 工具的 Tree-sitter 语义分析:

```
// Tree-sitter 解析命令行 → 抽取实际访问的目录路径
// cd /external/path && rm file → 触发 external_directory 权限请求
ctx.ask({
  permission: "external_directory",
  patterns: [resolvedPath],
  always: [resolvedPath],
})
```

维度	Claude Code	CoStrict
模式数量	6 种（含语义丰富的 plan/acceptEdits）	3 种（allow/ask/deny，更简洁）
规则来源	5 个来源，分级覆盖	多层配置聚合 + SQLite 持久化
Shell 静态分析	bashClassifier（强，ANT-ONLY，外部已 stub）	Tree-sitter 路径分析（原生开放）
YOLO 模式	bypassPermissions	Toggle + 状态持久化到 kv.json
Agent 隔离	Agent 级别权限继承	每个 Agent 独立 PermissionRuleset
规则持久化	会话内记忆（重启消失）	SQLite（跨会话记住）

3.5 多模型支持

维度	Claude Code	CoStrict
Provider 数量	4 个（Anthropic + 云厂商托管）	20+
格式转换层	无格式转换，直接透传	统一归一化层，消化各 Provider 格式差异
模型能力元数据	无	每模型标注工具调用/推理/附件/成本/限制
不支持工具的模型	无保护，可能发送无效调用	按能力动态决策，不发送工具定义
子 Agent 模型	限定 Claude 系列	任意 Provider + 模型自由组合
模型专属 prompt	无	支持按模型定制 system prompt

3.6 Agent 系统

3.6.1 Claude Code 的 AgentTool

关键文件：`src/tools/AgentTool/`

```
// 三种执行模式
// 1. fork 模式：继承父上下文，同步等待结果，独立对话历史
// 2. async 模式：后台运行，主流程继续
// 3. background 模式：完全独立，任务队列驱动

// 每个子 Agent 有：
// - 独立 AbortController（某个失败不影响其他）
// - 独立 MCP 服务器配置（agentDefinition.mcpServers）
// - 独立 turn 限制
// - 侧链存储（recordSidechainTranscript）
// - 嵌套深度追踪（queryTracking.depth）
```

Agent 间通信（`SendMessageTool`）：

```
to: "agent-name"        // 点对点
to: "*"                 // 广播给所有 Agent
to: "uds:<path>"        // Unix Domain Socket（本地 peer）
to: "bridge:<session>"  // 远程控制 peer（BRIDGE_MODE）
```

3.6.2 Claude Code 的任务系统（多 Agent 协调基础）

任务系统是 Claude Code 支撑多 Agent 协调的核心基础设施：

```
// 6 个工具：TaskCreate / TaskGet / TaskUpdate / TaskList / TaskOutput / TaskStop

// 任务认领（原子操作，文件锁保护）
claimTask(taskListId, taskId, claimantAgentId) {
  // 检查 blockedBy 依赖 → 写入 owner 字段
}
```

```
// Agent 退出时任务移交
unassignTeammateTasks(teamName, teammateId, reason) {
  // 重置 in_progress 任务为 pending，等待其他 Agent 认领
}

// DiskTaskOutput：异步任务输出写临时文件
// MAX_TASK_OUTPUT_BYTES = 5GB
// 增量轮询（每 100ms），读新增部分

// 完成通知以 XML 注入 Agent 的 user 消息
<task-notification>
  <task-id>...</task-id>
  <status>completed</status>
  <output-file>...</output-file>
</task-notification>
```

3.6.3 CoStrict 的 Agent 系统

关键文件：`packages/opencode/src/agent/`

```
Agent.Info = {
  name: string
  mode: "subagent" | "primary" | "all"
  permission: PermissionNext.Ruleset  // 每个 Agent 独立权限规则集
  model?: { modelID, providerID }     // 可用不同提供商的不同模型
  prompt?: string
  model_prompts?: Record<providerID, string>  // 提供商专属提示词
  steps?: number                       // 最大步数限制
  tools?: Record<string, boolean>      // 工具启用/禁用白名单
  temperature?: number
  topP?: number
}
```

17 个内置 Agent（双语 zh-CN + en）：

```
Wiki 文档系列（4个）：project-analyze / catalogue-design / document-generate /
index-generation
Plan 执行系列（5个）：plan-apply / plan-fix-agent / plan-quick-explore / plan-sub-
coding / plan-task-check
Spec 规格系列（5个）：spec-design / spec-plan-manager / spec-plan / spec-
requirement / spec-task
Strict 严格系列（2个）：strict-plan / strict-spec
TDD 系列（1个）：tdd
```

维度	Claude Code	CoStrict
执行模式	fork / async / background 三种	subagent / primary
上下文隔离	fork 完全隔离 + 侧链存储	独立 Session + SQLite 持久化
模型统一性	**所有子代理必须同一提供商**	**每个 Agent 可用不同提供商+模型**
Agent 间通信	SendMessageTool（点对点/广播）	无专用通信工具
多 Agent 协调	任务系统（文件锁 + 依赖图 + 认领）	无等效机制
内置 Agent	无（动态创建）	17 个预定义（覆盖研发全流程）
双语支持	无	zh-CN + en
嵌套支持	queryTracking.depth 追踪	支持

3.7 状态管理与持久化

3.7.1 Claude Code 的状态管理

`src/state/store.ts` **极简 Store**：

```
store = {
  getState(): AppState
  setState(updater: (prev: AppState) => AppState): void  // 强制不可变更新
  subscribe(callback: (newState, oldState) => void): () => void
}
```

AppState 包含：UI 状态 / 权限上下文 / MCP 连接 / 任务与代理 / 插件系统 / 通知队列 / Tmux 集成状态。

持久化：仅 JSON 文件（`~/.claude/settings.json`、任务文件系统 `~/.config/claude/tasks/`），进程级内存状态不跨进程恢复。

3.7.2 CoStrict 的 SQLite 持久化架构

核心表结构（Drizzle ORM 类型安全）：

```
ProjectTable  (id, directory, title)
SessionTable  (id, project_id, parent_id, slug, title, revert, permission,
               time_compacting, time_archived, summary_additions/deletions/files)
MessageTable  (id, session_id, time_created, data: JSON)
PartTable     (id, message_id, session_id, data: JSON)
  -- Part 类型 13 种: text / reasoning / tool / step-start/finish /
  --   patch / compaction / subtask / file / agent / snapshot / retry
TodoTable
PermissionTable (project_id, data: Ruleset JSON)  -- 持久化已批准的权限规则
```

Session Fork 能力（Claude Code 无）：

```
// Part 分片设计 → Session.fork() 可在任意消息节点分叉新会话
// UI 实时渲染通过 Bus.publish(Event.PartDelta) 增量推送
// 压缩精确标记哪些 Part 已被压缩（time.compacted 字段）
```

3.7.3 Git 快照系统（CoStrict 独有）

`packages/opencode/src/snapshot/index.ts`：

```
// 每个 LLM step 开始前自动创建快照（processor.ts 集成）
track()   → git write-tree → 返回树对象哈希
patch()   → git diff --name-only {hash}  → 收集文件变化
restore() → git read-tree + git checkout-index -a -f

// 磁盘空间保护
DEFAULT_MIN_FREE_SPACE = "5GB"    // 低于此值禁用快照
DEFAULT_CHECK_INTERVAL = 60 秒
// 定时清理：git gc --prune=7.days（每小时）
```

维度	Claude Code	CoStrict
状态存储	内存 + JSON 文件（进程退出即丢）	SQLite WAL（持久，可恢复）
会话历史	仅当前进程可见	SQLite，任意时刻可重建
Session Fork	无	任意消息节点分叉新会话
Git 快照	无	每 step 自动创建（git diff 低开销）
迁移系统	配置迁移脚本（`src/migrations/`，9个文件）	编译期嵌入，时间戳排序自动执行
Part 粒度	消息级别	13 种 Part 类型，细粒度分片
权限持久化	会话内记忆（重启消失）	PermissionTable（跨会话记住）

3.8 MCP 集成

3.8.1 传输层支持

Claude Code（`src/services/mcp/client.ts`）：

```
StdIO（本地子进程）
  → 不自动重连（子进程崩溃意味着更大问题）
SSE（单向流）
  → 长连接，不受 60 秒超时约束
  → 指数退避重连（1s→2s→4s→8s→16s，上限 30s，最多 5 次）
HTTP（Streamable HTTP）
  → 会话 404 + -32001 → 清除 memoize 缓存，强制重建连接
WebSocket（双向）
  → Bun 原生 / Node.js fallback，支持 mTLS
SDK Control Channel（进程内嵌入）
  → 通过控制请求/响应对通信，无网络开销
```

CoStrict（`packages/opencode/src/mcp/index.ts`）：

```
StreamableHTTP（优先）→ SSE（降级备选）→ Stdio（本地）
默认超时 30 秒，每个 Server 可独立覆盖
关闭时清理完整进程树（含 grandchild），避免孤立进程
```

3.8.2 OAuth 实现对比

Claude Code（`src/services/mcp/auth.ts`）：

```
// 令牌过期前 5 分钟主动刷新（不等 401 再补救）
// Step-up 认证：403 insufficient_scope → 强制重走完整 OAuth
// XAA（Cross-App Access）：利用缓存 IdP id_token 静默交换
// 令牌存储：系统 keychain（不写明文文件）
```

CoStrict（`mcp/index.ts`）：

```
// 回调地址固定：http://127.0.0.1:19876/mcp/oauth/callback
// CSRF 保护：crypto.getRandomValues(32字节) state 参数
// PKCE 流程：grant_types = ["authorization_code", "refresh_token"]
// 客户端 ID 三层优先级：配置指定 > 动态注册 > 新注册
// 令牌存储：~/.config/opencode/mcp-auth.json（权限 0o600）
```

维度	Claude Code	CoStrict
传输协议	5 种（StdIO/SSE/HTTP/WS/SDK-内嵌）	3 种（StdIO/SSE/HTTP）
重连策略	分协议差异化（StdIO 不重连，HTTP 有会话恢复）	统一超时管理
OAuth 支持	完整（2465 行，含 XAA 静默交换）	完整 PKCE + CSRF 保护
令牌存储	系统 keychain	文件（0o600 权限）
Step-up 认证	支持（insufficient_scope 自动升级）	无
工具转换	MCPTool 适配 + readOnlyHint→isConcurrencySafe	convertMcpTool + schema 强制
URL Elicitation	支持（-32042 弹对话框，最多 3 次）	无

3.9 LSP 集成（CoStrict 独有）

CoStrict 实现了完整的 LSP client/server 双端（`src/lsp/`）：

```
// LSP 客户端能力
LSP.diagnostics    → 实时错误/警告反馈给 Agent
LSP.hover          → 精确类型信息（无幻觉）
LSP.definition     → 精确定义位置
LSP.references     → 所有引用
LSP.implementations → 接口的全部实现
LSP.rename         → 语义级重命名（不是文本替换）
LSP.completion     → 上下文感知补全
LSP.documentSymbol → 文件内符号（配合 file-outline 使用）
```

自动下载管理的 LSP Server（4 种）：

```
clangd      → C/C++
JDTLS       → Java
rust-analyzer → Rust
gopls       → Go
```

3.10 配置系统

3.10.1 Claude Code 的配置层次

```
/etc/claude-code/CLAUDE.md     ← 企业托管（最高优先级，不可覆盖）
~/.claude/CLAUDE.md             ← 用户全局
~/.claude/settings.json         ← 全局设置
{project}/CLAUDE.md             ← 项目根
{project}/.claude/rules/*.md   ← 规则文件集
{project}/.claude/settings.json ← 项目设置
{project}/CLAUDE.local.md      ← 本地私有（不提交 git）

managed-mcp.json（企业托管 MCP 配置）
```

Claude Code 还有 `src/migrations/`（9 个迁移文件），启动时自动执行版本升级（如 Sonnet 4.5→4.6 别名迁移）。

3.10.2 CoStrict 的七层配置优先级（从低到高）

```
1. 远程 .well-known/opencode     ← 组织级默认（最低）
2. 全局用户配置                  ← ~/.config/opencode/opencode.json
3. OPENCODE_CONFIG 环境变量       ← 自定义配置路径
4. 项目配置                      ← opencode.json / costrict.json
5. .opencode / .costrict 目录    ← agents/ commands/ plugins/ tools/
6. OPENCODE_CONFIG_CONTENT        ← 内联配置
7. 企业托管配置                  ← /etc/opencode（最高）
                                      /Library/Application
Support/opencode（macOS）
                                      %ProgramData%/opencode（Windows）
```

配置聚合策略（`mergeConfigConcatArrays`）：plugin[] 和 instructions[] 合并去重，其他字段 mergeDeep。

目录配置支持动态依赖安装（`needsInstall + installDependencies`）。兼容 OpenCode 配置格式（`Flag.COSTRICT_ENABLE_OPENCODE_CONFIG`）。

维度	Claude Code	CoStrict
配置层数	约 5 层	7 层
企业托管	`/etc/claude-code/CLAUDE.md`	`/etc/opencode`（完整 JSON 配置）
组织默认	无	`.well-known/opencode`
格式兼容	CLAUDE.md + JSON	OpenCode + CoStrict 双格式
动态依赖	无	needsInstall 检测 + 自动安装
版本迁移	9 个迁移脚本（启动自动执行）	数据库迁移（编译期嵌入）

3.11 扩展性：Plugin / Hook 系统

3.11.1 Claude Code 的 Hook 机制

5 种 Hook 时机（`src/hooks/`）：

```
pre_tool_use          ← 工具执行前（可修改输入或拦截）
post_tool_use         ← 工具执行后（格式化、检查）
permission_request    ← 权限决策前（外包给合规脚本）
post_compact          ← 压缩完成后（通知 Slack 等）
stop                  ← 会话结束时
```

```
// 配置示例
{
  "hooks": {
    "post_tool_use": {
      "prettier": "prettier --write {FILE}",
      "tsc": "tsc --noEmit"
    },
    "permission_request": "scripts/approve-policy.sh {TOOL} {INPUT}"
  }
}
```

Hook 脚本退出码 2 = 阻塞（BlockingError），0 = 通过。

3.11.2 CoStrict 的 Plugin 事件系统

8 个 Hook 事件点（`src/plugin/index.ts`）：

```
"chat.params"                          // LLM 调用参数（temperature/topP）
"chat.headers"                         // HTTP 请求头定制
"experimental.chat.system.transform"   // 系统提示词变换
"experimental.chat.messages.transform" // 消息列表变换（发送前）
"experimental.text.complete"           // 文本输出完成后处理
"experimental.session.compacting"      // 压缩提示词注入/替换
"experimental.shell.env"               // Shell 环境变量定制
"tool.execute.before"                  // 工具执行前干预（可修改参数或阻断）
```

内置 Plugin 列表：

```
const INTERNAL_PLUGINS = [
  CodexAuthPlugin,     // OpenAI Codex OAuth（src/plugin/codex.ts）
  CopilotAuthPlugin,   // GitHub Copilot 鉴权
  CoStrictAuthPlugin,  // CoStrict 平台鉴权（含动态 token 刷新）
  TDDPlugin,           // TDD 工作流增强
  GitlabAuthPlugin,    // GitLab 鉴权（@gitlab/opencode-gitlab-auth）
  LearningPlugin,      // 学习系统（自动规则提炼）
]
```

外部 Plugin 动态安装:

```
// config.json
{ "plugin": ["my-custom-plugin@1.0.0", "file://./local-plugin"] }
// Plugin.list() 通过 BunProc.install() 自动安装 npm 包
// 支持 file:// 协议的本地插件
```

3.12 UI 与交互体验

3.12.1 Claude Code 的 REPL（`src/screens/REPL.tsx`）

```
VirtualMessageList      → 虚拟化滚动（大量消息不卡顿）
推测执行架构            → overlay 预测下一步工具调用（隐藏延迟）
多模式支持              → bash / plan / normal
权限对话                → PermissionRequest + ElicitationDialog + PromptDialog
后台任务面板            → LocalShellTask + InProcessTeammateTask 导航
Tmux 集成               → Tungsten 原生桥接
浏览器工具              → Bagel 集成
远程会话                → Bridge（BRIDGE_MODE，ANT-ONLY）
Swarm 团队协作          → ANT-ONLY
FPS 指标收集
成本追踪与警告
Vim 模式                → src/vim/（motions/operators/textObjects/transitions）
伴侣系统（Buddy）       → 面向 claude.ai 订阅用户的趣味角色
```

3.12.2 CoStrict 的 TUI + Desktop

```
TUI（Ink）：多 session 管理，基础任务面板
Desktop（Electron，packages/desktop-electron）：
  - 文件树 GUI
  - 模型选择界面
  - Provider 配置向导
  - 与 Agent 核心共享同一 Provider 层
Web App（SaaS 版）：多人协作，与 TUI 同步
```

维度	Claude Code	CoStrict
消息渲染	虚拟化列表（高性能）	标准渲染（TUI）
推测执行	有（overlay 预测）	无
Tmux 集成	Tungsten（ANT-ONLY）	无
Vim 模式	完整（5 个文件，全操作符）	无
桌面应用	无（纯 CLI）	Electron 桌面端
Web 协作	Bridge（ANT-ONLY）	Web App（SaaS，多人）
多端一致性	单一 CLI	CLI / Desktop / Web 共享 Agent 核心
趣味功能	Buddy 角色系统	无

3.13 构建时特性门控

3.13.1 Claude Code 的 bun:bundle 机制

```
// src/main.tsx（正式构建）
import { feature } from 'bun:bundle'
// 编译时将 feature('FLAG') 替换为 true/false 常量
// false 分支在 tree-shaking 阶段完全消除，不进入产物

// src/entrypoints/cli.tsx（反编译版 polyfill）
const feature = (_name: string) => false  // 所有特性关闭
```

已识别的 ANT-ONLY 特性标志（部分）：

特性标志	功能
`COORDINATOR_MODE`	多 Agent 协调/Swarm
`BRIDGE_MODE`	IDE 远程控制桥接
`REACTIVE_COMPACT`	反应式上下文压缩
`CONTEXT_COLLAPSE`	上下文折叠（PTL 恢复）
`TOKEN_BUDGET`	Token 预算控制
`VOICE_MODE`	语音输入
`EXTRACT_MEMORIES`	自动记忆提取
`DAEMON`	后台守护进程
`PROACTIVE`	主动建议模式
`WORKFLOW_SCRIPTS`	工作流脚本工具

效果：ANT-ONLY 功能在发布包中物理不存在（tree-shaking 消除），反编译版与正式版行为差异来源于此。

3.13.2 CoStrict 的 Flag 系统

CoStrict 用运行时 `Flag` 系统做特性开关，特性代码在包中存在但被禁用——理论上可被外部启用，包体积略大。这是 CoStrict 相比 Claude Code 的架构劣势之一。

维度	Claude Code	CoStrict
门控时机	构建时（bun:bundle tree-shaking）	运行时（Flag 系统）
ANT-ONLY 功能可见性	物理不存在于发布包	代码存在但被禁用
包体积	特性代码完全剔除（25MB 精简）	包含所有特性代码
外部可激活	不可能（代码不存在）	理论上可激活

3.14 综合分析

3.14.1 两者的结构性定位

Claude Code 的结构性优势（难以被追赶）：

1. Anthropic 官方支持，内部版功能在正式版中已启用，能力远超公开版所见
2. 推测执行架构：感知延迟最低，流式输出第一个工具调用即调度
3. 五层上下文压缩体系：专为长任务设计，GC 集成防内存泄漏
4. 多 Agent 协调基础设施完整：原子认领 + 依赖图 + 多种通信方式 + 任务移交
5. 权限系统深度：6 种模式，预览模式先预览后执行，23 类危险模式静态检测
6. 构建时特性门控：内部功能代码物理不存在于发布包

CoStrict 的结构性优势（Claude Code 无法复制）：

1. 真正的模型无关：20+ Provider 开箱即用，商业模式决定 Claude Code 永远只做 Anthropic
2. 17 个工作流 Agent：软件工程方法论固化为可执行流水线
3. 针对每个开源模型的已知弱点定向优化（模型专属 system prompt）
4. LSP + Tree-sitter：工具替代模型推理，不依赖模型猜测
5. 数据库 + 快照 + Checkpoint 三重状态保障：开源模型生产环境使用的关键
6. 事件总线 + 插件系统：零侵入扩展，9 个生命周期钩子

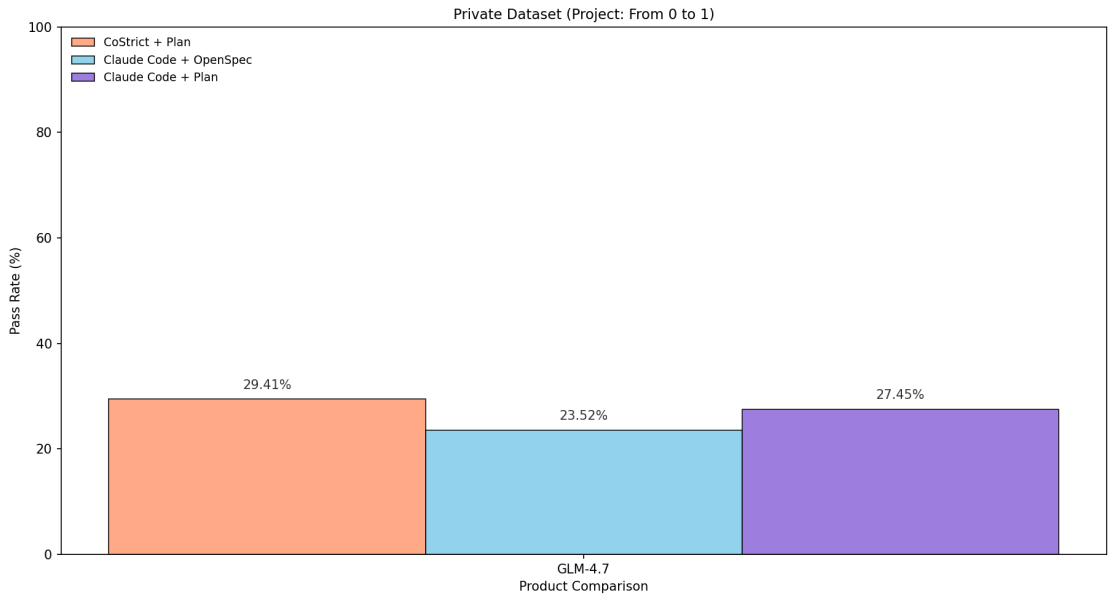
3.14.2 一句话定位

Claude Code：把智能集中在模型本身，工具层为 Claude 系列深度优化，五层压缩 + 流式调度 + 多 Agent 协调基础设施让复杂长任务可靠执行，适合以 Claude 为核心的高强度编程场景。

CoStrict：把智能分散到平台层，ProviderTransform + LSP + 工作流 Agent 补偿模型短板，模型能力越弱于 Claude，平台层的补偿效果越显著，适合多模型混用、企业合规、大型代码库场景。

四、CoStrict+开源模型 比 claude code+开源模型 更强?

内部测评数据，这里以GLM-4.7为例，后续会补充更多模型：



4.1 源码层面：客观能力对比

> **结论先行**：基础架构层面，Claude Code 具有明显的工程积累优势，CoStrict 整体偏弱。CoStrict 的核心竞争力不在这一层——在第二部分的工程方法论固化。

Claude Code 的结构性优势（基础能力层）

以下 6 个维度 Claude Code 有明显优势，CoStrict 短期内难以追赶：

① 工具并发执行：自研调度器 vs 委托 SDK

Claude Code 在收到第一个工具调用指令时立即开始执行，无需等待完整输出；并发安全的连续工具合并为一批并行执行（上限 10 个）；某工具失败时立即取消同批次其他任务。

CoStrict 把多步工具执行委托给第三方 AI SDK，调度时机由 SDK 内部控制，无法精细调优。

CoStrict 劣势：延迟更高、并发精度弱、错误传播路径不透明。

---

② 上下文压缩体系：五层 vs 三路

Claude Code 五层叠加（静态记忆 + 动态阈值 + 双路压缩执行 + 零 LLM 清理 + GC 集成）。CoStrict 只有三路（工具输出裁剪 / 会话压缩 / 溢出压缩），无零 LLM 清理，无 GC 集成，无断路器。

CoStrict 劣势：压缩层次少、触发精度低、长对话内存压力缺乏针对性处理。

---

③ 多 Agent 协调基础设施

Claude Code 提供完整的任务工具集、原子任务认领、依赖图调度、多种通信方式、大容量输出支持以及 Agent 退出时任务自动移交。

CoStrict **没有等效的任务认领/依赖图/Agent 间通信基础设施。**

CoStrict 劣势：无法构建多 Agent 真正并行协作的工作流。

---

④ 权限系统深度：6 种模式 vs 3 种

Claude Code 含预览模式（写操作先返回预览，用户确认后再执行）和自动接受文件修改模式（Shell 命令仍需确认）。CoStrict 只有三种动作（允许/询问/拒绝），无预览模式语义）。

⑤ Bash 安全检查深度

Claude Code 静态检测 23 类提示注入模式。CoStrict 进行路径分析，防路径遍历和符号链接攻击，不含提示注入静态检测。

---

⑥ UI 工程完整度

特性	Claude Code	CoStrict
Vim 模式	完整支持	无
推测执行	预测下一步工具调用	无
消息渲染	虚拟化列表，大量消息不卡顿	标准渲染
Jupyter 支持	支持	无

CoStrict 的局部优势（仅接入非 Anthropic 模型时显现）

以下优势**仅在接入 DeepSeek、Qwen、Llama 等非 Anthropic 模型时才体现**：

① **Provider 适配层**：开源模型接入开箱可用，Claude Code 格式直接透传可能不兼容。

② **模型能力感知**：不向不支持工具调用的模型发送工具定义，避免无效调用。

③ **代码智能工具**：AST 结构提取、结构化推理等工具，Claude Code 均不具备。

④ **Agent 异构模型支持**：每个子 Agent 可指定不同 Provider + 模型，构建多模型混合工作流。

⑤ **数据库持久化**：开源模型推理慢，长任务崩溃代价更高，持久化价值更明显。

结论：基础架构 CoStrict 整体弱于 Claude Code

直接说：CoStrict 的基础架构稍微差点。工具并发调度、上下文压缩体系、多 Agent 协调基础设施、权限系统深度——Claude Code 在这几个维度是明显领先的。

CoStrict 在 Provider 适配、AST 代码工具、异构模型支持上有局部优势，但这些优势**在使用 Claude 模型时几乎不体现**，只在接入开源模型时才显现。

CoStrict 的核心竞争力不在基础架构层，在第二部分——把工程方法论固化进可执行的工作流流水线。

4.2 内置工作流增强（CoStrict 核心竞争力）

> CoStrict 相比 Claude Code 最不可替代的差异在这里——将软件工程方法论固化为可执行的工作流流水线，覆盖项目初始化、代码探索、需求澄清、代码审查、测试校验五个环节，不依赖模型能力，用流程弥补推理短板。

4.2.1 /init 增强：enhanced-initialize → AGENTS.md

Claude Code 的 /init：

- 旧版：单段提示词，模型自由发挥，无结构约束
- 新版：8 阶段流程，但属于内部功能，外部用户需手动开启环境变量
- 产物：CLAUDE.md，针对 Claude 指令格式优化，无 Git 历史分析，无质量门禁

CoStrict 的 /init：
产物是 `AGENTS.md`——供**任意 AI Coding Agent** 使用的标准化项目文档，分 5 个串行子任务：

- **子任务 2**（差异最大）：分析近 6 个月 Git 提交历史，按目录层级聚合修改记录，统计目录修改次数，8 级优先级展示目录树，总行数 ≤150 行，标准树形符号，每条描述 ≤30 字
- **子任务 3**：分析近 3 个月 Git 提交历史提炼编码规范，总条数 ≤50 条，单条 ≤50 字，必须使用「必须/禁止/应」等强约束词
- **子任务 5**：5 维度审查门禁（文档完整性 + 格式合规性 + 内容准确性 + 指导性 + 篇幅合规）→ 最小改动修正 → 二次复核

维度	Claude Code /init	CoStrict /init
产物	CLAUDE.md（供 Claude 读）	AGENTS.md（供任意 AI Agent 读）
执行结构	单段提示词，模型自由发挥	5 个串行子任务，每任务委派子 Agent
Git 历史分析	无	近 6 个月目录修改频率 + 近 3 个月规范提取
质量门禁	无	5 维度审查 + 最小改动修正 + 二次复核
外部可用状态	旧版（无结构）/ 新版需环境变量	全用户开放

4.2.2 Explore 能力增强：QuickExplore Agent

Claude Code 没有独立的探索 Agent，主模型直接在主会话中探索，消耗主会话 token，无结构化约束。

QuickExplore 是专用只读探索 Agent，在**独立上下文**中运行：

漏斗式收敛策略（宏观 → 微观四级）：

- 目录（限定到 2-3 级）→ 文件（按类型过滤）→ 骨架（AST 查看结构）→ 代码片段（只读必要行号范围，超 500 行必须指定范围）
- 禁止大范围全局检索

Git 历史挖掘（Claude Code 无）：搜索相关功能提交历史，查看变更差异提取可复用方案；搜索缺陷修复提交，提取已踩过的坑；追踪依赖历史变更，识别兼容性风险。

执行约束：控制在 30 轮内，连续 3 轮无进展立即调整策略，禁止修改任何代码。

输出证据要求：代码定位必须提供文件路径 + 行号 + 代码片段，Git 历史必须提供提交哈希 + 日期 + 变更摘要。

4.2.3 需求澄清工具体系

Claude Code 没有独立的需求澄清机制，是否提问完全由模型自主判断。

StrictPlan 三条铁律：探索驱动澄清 + 项目信息优先 + 代码可答则不问

需求复杂度感知：

- 需求详尽（用户提供文档/长段说明）→ 大幅减少提问
- 需求简短（一句话）→ 适度增加提问
- 需求已明确的细节 → 禁止重复提问

Requirement Agent 9 阶段流程：5W2H 分析法（三层目标挖掘）→ 需求澄清（最多 3 轮，总计 ≤15 个问题）→ INVEST 原则用户故事 → 格式化系统需求 → 成功标准定义 → SMART 检验法质量审查。

4.2.4 ReviewAndFix Agent

定位：执行完成后的代码审查与修复环节。不直接修改代码——只修改任务计划文档，代码修改必须委托子编码 Agent 执行。

工作流程（4 阶段）：

1. 读取任务文档 + 查看已完成进度快照，了解当前状态
2. 分析反馈 → 在任务文档末尾插入新修复任务 → 用户确认
3. 并行/串行分发给子编码 Agent，四维度审查（最小变更 / 风格一致 / 架构尊重 / 任务完整）
4. 所有任务完成后向用户确认

维度	Claude Code	CoStrict ReviewAndFix
代码修改方式	主模型直接修改	任务文档驱动，代码由子 Agent 执行
代码审查	无	快照差异 + 4 维度审查标准
架构保护	依赖模型判断	明确约束：遵循项目既有目录结构和设计模式

4.2.5 测试校验：逐级校验流水线

Claude Code 没有内置测试工作流 Agent，无结构化阶段划分，无自动修复机制。

CoStrict 四级校验流水线：

- **Level 0：TestPrepare** → 只做命令定位，不执行任何命令，填充测试指南文档
- **Level 1：RunAndFix** → 编译/构建/类型检查验证（不涉及业务逻辑测试）
- **Level 2：TestDesign** → 测试用例设计（正常/边界/异常三类场景）
- **Level 3：TestAndFix** → 测试执行 + 自动修复（最多 3 轮）

RunAndFix 关键约束：

- ☑ 可修复：语法错误、类型错误、缺失导入、编译错误
- ✕ 不修复：测试失败、缺失依赖、环境问题
- ✕ 严禁：运行测试命令、git 历史回滚操作
- 文件修改权限区分：最近修改的文件自动修复，非最近修改文件需用户授权

维度	Claude Code	CoStrict 测试流水线
测试工作流	无内置工作流	4 级协调
规范文档	无	测试指南文档
编译与测试分离	不区分	编译问题与测试问题分层处理
文件修改权限	无区分	最近修改文件自动修复，非最近文件需用户授权
自动修复	无	最多 3 轮
用户代码保护	无	明确禁止 git 回滚操作

4.3 小结

基础架构层的结构性对比，Claude Code 的结构性优势（难以被追赶）：

1. Anthropic 官方支持，内部版功能在正式版中已启用，能力远超公开版所见
2. 推测执行架构：感知延迟最低，流式输出第一个工具调用即调度
3. 五层上下文压缩体系：专为长任务设计，GC 集成防内存泄漏
4. 多 Agent 协调基础设施完整：原子认领 + 依赖图 + 多种通信方式 + 任务移交
5. 权限系统深度：6 种模式，预览模式先预览后执行，23 类危险模式静态检测
6. 构建时特性门控：内部功能代码物理不存在于发布包

工作流层的核心结论：以上五个工作流增强（/init → AGENTS.md、QuickExplore 漏斗收敛、探索驱动需求澄清、ReviewAndFix task.md 驱动、四级测试流水线）的共同特征是——**不依赖模型能力，用结构化流程弥补推理短板。**Claude Code 把质量寄托在模型上，模型强它就强；CoStrict 把方法论固化进平台层，任何模型都能得到结构化保障。

五、Claude Code 源码泄漏事件后，CoStrict 应该怎么做

5.1 差距全景

这次 Claude Code 源码泄漏，终于可以精确看清对手的实现细节，知道哪里真强、哪里是空壳。

核心能力对比矩阵：

#	核心能力	Claude Code	CoStrict	谁领先
1	**长任务可靠性**	五层压缩 + PTL 恢复链 + compact_boundary GC；进程内存不随对话无限增长	三路压缩 + SQLite 持久化（进程崩溃可恢复）	CC 更抗长任务崩溃，CoStrict 更抗进程崩溃
2	**工具调度效率**	流式输出第一个 block 立即调度，isConcurrencySafe 批分区，sibling abort	委托 Vercel AI SDK，调度时机由 SDK 控制	CC 领先（用户感知延迟显著更低）
3	**Agent 编排能力**	fork/async/background 三模式 + 6 工具任务系统 + SendMessage 广播	17 个预定义工作流 Agent，每个 Agent 可异构模型	CC 基础设施更强，CoStrict 工作流固化更好
4	**权限安全体系**	6 种模式（含 plan 模式预览）+ 5 来源分级	3 种 action + tree-sitter 路径分析 + SQLite 跨会话权限记忆	CC 模式更完整，CoStrict Bash 分析实际更可用
5	**多模型适配**	4 个提供商，绑定 Anthropic	20+ 提供商 + ProviderTransform + model_prompts	CoStrict 结构性领先（CC 永远不会追赶）
6	**代码理解精度**	全靠模型读文件	Tree-sitter AST + 完整 LSP client/server	CoStrict 结构性领先
7	**状态可靠性**	内存状态，进程退出即丢；无 Session Fork	SQLite WAL + Session.fork() + 每 step 自动 git 快照	CoStrict 结构性领先
8	**扩展与集成**	5 种 Hook	8 个生命周期钩子 + 6 内置 Plugin + npm 动态安装	CoStrict 领先

5.2 战略约束：为什么不能全面追赶

fork 策略的代价：CoStrict fork 自 OpenCode，上游同步是生命线。架构层改动越大，合并上游代码的难度呈指数级上升：改 100 行合并可能 1 小时，改 5000 行可能 1 周还合不完。

差距分类决策：

决策	差距项	理由
等待观望	工具调度延迟 / Agent 并发协调 / 上下文压缩保真度	需要绕过 Vercel AI SDK 封装，涉及多层联动，自行实现后合并上游代价极高。等 OpenCode 2-4 周
可纳入计划	Worktree 工具化	应用层改动，模块已有，直接暴露即可
长期跟进	长任务内存控制 / CLI 交互体验	compact_boundary + GC 联动需要 processor/storage 协同改造，优先级低
不追赶	Jupyter Notebook / 权限系统完整性 / 推测执行/Vim 模式	需求场景与主要用户群重合度低；与多端战略方向不同

为什么"等 OpenCode"是合理的：OpenCode 对 Claude Code 源码泄漏大概率会响应，如果上游改进了权限系统和多代理并发，CoStrict 直接合并即可。时间窗口：**观察 2-4 周**，看社区 PR 动向。

5.3 CoStrict 的差异化能力

能力	说明	对标 Claude Code
StrictPlan（SpecPlan / TaskPlan）	结构化任务规划，复杂需求拆解为可执行步骤	CC 无等价物，依赖模型自行规划
QuickExploreAgent	快速代码理解，专为探索场景优化的轻量级代理	CC 的 AgentTool 是通用代理，无专门探索模式
FixAgent 体系（RunAndFix / TestAndFix / ReviewAndFix）	自动诊断-修复闭环，覆盖运行、测试、审查三个场景	CC 无等价物，修复靠用户手动引导
sequential-thinking	结构化推理工具，支持分支思考和修订	CC 依赖模型原生 extended thinking
file-outline + LSP	语义级代码理解，Tree-sitter AST + LSP 双端	CC 完全没有，100% 依赖模型理解代码结构
多模型 + 模型专属 Prompt	20+ 提供商 + 模型专属 prompt	CC 锁定 Anthropic，商业模式决定无法改变

5.4 这些优势面临的风险

Claude Code 生态正在快速补齐短板：

- `superpowers`（skill 框架 + 20+ 预置 skill）

- `claude-mem`（外部记忆管理，弥补 CC 原生记忆能力不足）
- `everything-claude-code`（社区资源聚合，降低使用门槛）
- Skills Hub、OMC 等社区工具

核心判断：**CoStrict 内置的差异化能力，可能被 Claude Code 生态中的第三方工具抹平。**但"内置开箱即用" vs "自行安装配置"之间仍有体验差距，这个差距就是窗口期。

应对策略：借用开源最佳实践，内置到 CoStrict 开箱即用：

社区最佳实践	内置方式	体验优势
superpowers skills 等	已内置核心 skill（TDD、debugging 等）	零配置可用
结构化规划模式	StrictPlan 已原生实现	比 skill 更深度的集成
代码审查工作流	ReviewAndFix 自动闭环	不仅审查，还自动修复

5.5 核心判断

① 不盲目追赶 Claude Code 的架构优势。fork 策略决定了架构层改动必须谨慎，等 OpenCode 上游响应是当前最优选择。

② 上下文压缩是被低估的差距。CC 的五层叠加压缩不只是数量差异：compact_boundary GC 集成、Microcompact 零 LLM 清理、断路器保护是长任务可靠性的关键。CoStrict 的 compaction replay 机制是独特优势，应继续保留。

③ 推测执行 UX 差距是用户能感知的体验鸿沟。CC 的 StreamingToolExecutor 流式即调度是实际体验中最明显的差距，是 CoStrict 当前最大的 UX 短板。

④ CoStrict 的差异化能力（StrictPlan、QuickExplore、FixAgent、sequential-thinking）是真正的竞争力。持续深化，而不是把精力分散到追赶架构差距上。

⑤ 国产 AI Coding 工具的基本能力即将被磨平。Claude Code 源码泄漏会加速这个过程。竞争焦点将转移到开箱即用体验、企业级特性——这些正是 CoStrict 的布局方向。

⑥ vscode/jetbrains 插件端差距巨大。当前选择的策略是挑选问题 TOP 10 进行处理；CLI/Web 端是面向未来的主战场，后续能力更多基于 CLI 构建。